

Project - Build A Math Tutor

Mrs. Ramsey-Rutledge

September 2026

1 Introduction

1.1 Overview

You will create a Java program called **Math Tutor**. The program will act as a simple tutoring tool that can solve several types of math problems you have learned in previous mathematics courses. Your program should allow the user to choose a problem type, enter values, and receive a calculated answer.

The goal of this project is to practice:

- User input
- Variables and data types
- Mathematical operators
- The Java **Math** class
- Methods
- Program organization
- Testing and debugging

This project is less about advanced mathematics and more about using Java to solve familiar problems. Focus on writing clear, organized, and well-tested code.

1.2 Requirements

Solve At Least Three Different Types of Problems

Your tutor must be able to solve **at least three** different kinds of math problems.

Possible examples include:

1. Calculate the solution to Quadratic formula
2. Given two sides of a right triangle, calculate the third side
3. Calculate area of a circle
4. Calculate the final amount after interest is applied
5. Calculate slope between two points

You may choose other math topics as well. Choose these problems carefully as you are required to use at least four math operators. These include modulo, addition, subtraction, division, multiplication, exponents, roots, derivatives, and many more.

Use User Input

Your program must ask the user for information. The program should prompt the user for all values needed to solve the selected problem. Please use **scanner** for reading inputs.

Example:

```
Welcome to Math Tutor!
```

```
Choose a problem:
```

1. Circle Area
2. Pythagorean Theorem
3. Quadratic Formula

Use the Math Class

Your program must use the Java **Math** class at least once.

Examples:

```
Math.sqrt()
Math.pow()
Math.PI
Math.abs()
```

Create At Least Three Methods

Your program must contain at least three methods besides **main()**. Each method should solve one type of problem. Additionally, these methods should be housed in their respective classes and files.

Examples:

```
public static void solveCircle()
public static void solveDistance()
public static void solveQuadratic()
```

1.3 Grading

This project will be graded in two parts. First, the student should complete the writeup by providing the expected student responses to each part during the coding process. The writeup should be created by modifying this tex file using Overleaf. The student will submit the writeup as a pdf. Second, the student must submit the code in a zip folder. The teacher will test run the code and verify that it matches the project description. The possible points are broken down in the following table.

Category	Possible Points	Earned Points
1 Introduction	5	
2 Setting up Github	5	
3 Customize VS Code	5	
4 UML Diagrams	10	
5 Accepting user input	10	
6 Creating your first class	10	
7 Completing the other two classes	10	
8 Verifying your code	15	
10 Uploaded code	5	
Code compiles and runs	15	
Code organization and readability	10	
Total	100	

Extra Credit is offered by completing additional challenges as outlined in section 9. Each challenge successfully implemented is rewarded with 2 extra credit points. Successfully completing all 7 challenges could result in a maximum score of 114/100.

Projects submitted that use generative AI will result in an email being sent home and an automatic zero on the project. If AI is suspected to be used, the teacher will contact the student first. The teacher may ask the student to recreate the program or a similar problem in front of the teacher.

1.4 Student Acknowledgment

For the student response to part I, please write in the box "I, (Your Name), acknowledge the overview of this assignment and how I will be graded. I will produce a Java project that is one hundred percent my own work and I acknowledge any consequences of cheating."

Solution: I, Charlotte Collard, acknowledge the overview of this assignment and how I will be graded. I will produce a Java project that is one hundred percent my own work and I acknowledge any consequences of cheating.

2 Setting up Github

For more detailed instructions, access the instructions at [Setting Up a Github Account](#).

1. Open [Github.com](#)
2. Type in your email and select Sign up for Github
3. Create a password and username before clicking Create account
4. Click your profile picture in the upper right corner and then select Profile
5. Select Edit profile to add a name and bio
6. Save your changes

For the student response, please **provide a screenshot of your profile**. For full points, add your name or pseudonym and a biography. If you use a pseudonym, please include it written out in your answer.

Solution:

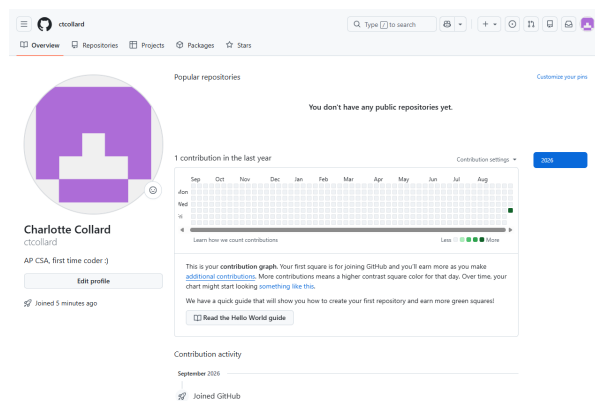


Figure 1: Screenshot of My Profile

3 Customize VS Code

Visual Studio Code (VS Code) is the integrated developer environment (IDE) of choice for this class. VS Code can host several different languages based on the extensions you install. For this class, you will need to modify VS Code to compile Java projects.

1. Open VS Code
2. Sign in with Github
3. Click the Extensions icon on the left side or press `ctrl`+`shift`+`X`
4. Search Github Copilot and disable AI features (Workspace)
5. Search for **Extension Pack for Java** and install it
6. Search for **Github Pull Requests and Issues** and install it
7. Open settings and disable AI code suggestions and completions
8. Close VS Code and reopen it
9. In File Explorer, create a folder labeled "APCSA"
10. In VS Code, hit File and Open Folder
11. Open the APCS A folder
12. In the terminal, type

```
cd .\APCSA\
```

Note that `tab` can autocomplete the name.

13. Screenshot your whole desktop to show VS code installed and your APCS A folder open

You will need to submit the screenshot in your writeup.

Solution:

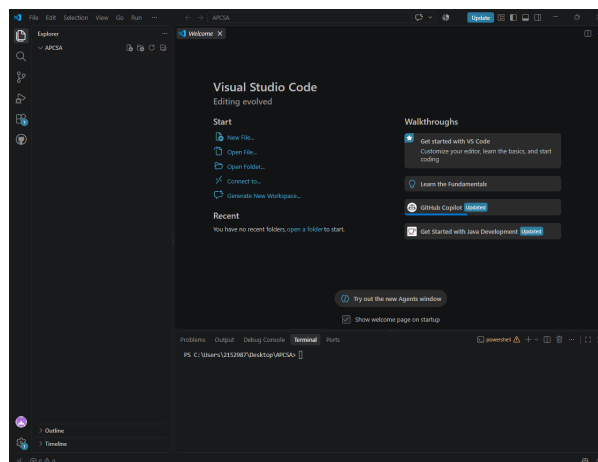


Figure 2: This image shows the installed VS code with my APCS A folder open.

4 UML Diagrams

UML Diagrams are a standardized visual blueprint, used to design, map, and communicate the attributes and behaviors of a class. For this project, you will create three classes that represent each math problem that you choose. Each of these classes will have both members and methods. On paper, google slides, or a word processor, you will want to plan out diagrams for your three classes. Be sure to distinguish the scope, any return types, data types, and names for all behaviors or attributes. You will submit these separately from the write up.

5 Accepting User Input

First, in Github, create a Repository. Give it a descriptive name and turn off the README. Copy the link for the repository. In VS Code, navigate to your "APCSA" folder. In the terminal, type

```
git clone <url>
```

where <url> is replaced with the link to your repository. You will need to navigate to the repository folder using `cd <repository name>` and the name of your folder. `tab` will autocomplete the name. Next, create a driver file. You can call it "MathTutor.java" or something else along those lines. In this file, create the framework for your main.

```
1  /*
2  * Title of Project
3  * Author's Name
4  * Purpose
5  *
6  * Resources:
7  *
8  */
9
10 import java.util.Scanner;
11
12 public class MathTutor
13 {
14     public static void main(String[] args) throws IOException
15     {
16
17     }
18 }
```

At the top of each file, I expect a brief that includes a title, author's name, purpose, and resources you used.

You may notice that the Scanner library is already imported. This library is helpful for reading inputs. Check out the details for this library [here](#). You must first instantiate a Scanner object. Next, you will need to use print statements to prompt the user to pick one of three options. Then use the Scanner object to collect the input. If you use any resources, add that to the header and how you used it as a reference.

To run your code, click the small Run button in the top left corner of the code editor and select Run without debugging. You can also select the ▶ icon in the top right corner to run.

While working on your code, save your progress locally with the Save button then update your Github repository using the commands

```
git add .
git commit -m "This is an example edit"
git push
```

Make sure to change what is in quotations because this is the name of your commit. It should be descriptive of what the change you made do. If you see the message,

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Please save your changes locally. The saved changed files will appear in a different color when they do not match the remote repository.

To document progress on this step, **include a recent screenshot of the repository you created**. This screenshot should show your latest commit. Additionally, please **include the code you wrote to accept user input**.

Solution:

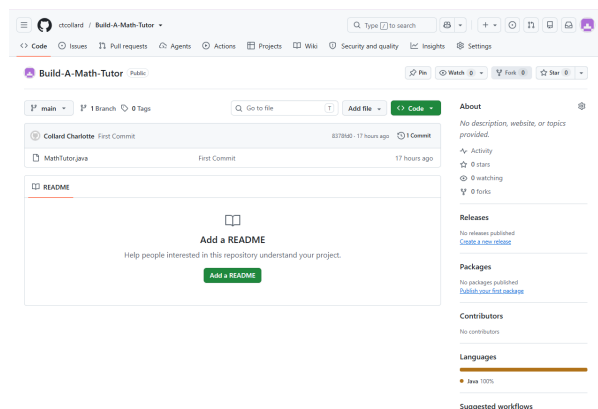


Figure 3: Screenshot of my repository

```

1  import java . util . Scanner ;
2  public class MathTutor {
3      public static void main ( String [] args ) throws IOException
4  {
5      Scanner input = new Scanner(System.in);
6      System.out.println("Welcome to Math Tutor!" + "\n");
7      System.out.println("Choose an option: Pythagorean theorem (enter 1), area
8      of a circle (enter 2), or slope of a line between two points (enter 3)."+ "\n");
9      int userInput = input.nextInt();
10     if ( userInput ==1) {
11         PythagoreanTheorem problem = new PythagoreanTheorem () ;
12         problem . solver () ;
13     }
14     if ( userInput ==2) {
15         AreaOfCircle problem = new AreaOfCircle () ;
16         problem . solver () ;
17     }
18     if ( userInput ==3) {
19         SlopeOfLine problem = new SlopeOfLine () ;
20         problem . solver () ;

```

```

20 }
21 }
22 }

```

6 Creating your first class

If you have not already, you will want to draft UML diagrams for each class. These classes will encapsulate the code required for your three problem choices. These diagrams should include the name of the class, attributes, and behaviors.

First, you will want to create a new file with the name of your first problem type. For example, the following code will use the file name "Problem1.java", but you could use "AreaOfCircle .java" or "CalculateDistance.java". The following code provides an example of the class layout with private members, a constructor, and a method. This example code uses the constructor to request additional inputs.

```

1  /*
2  * Title of Class
3  * Author's Name
4  * Purpose
5  *
6  * Resources:
7  *
8  */
9
10 import java.util.Scanner;
11
12 public class Problem1
13 {
14     private double input;
15     // add other private members if necessary
16
17     public Problem1() {
18         System.out.println("Please input a value: ");
19         // add your code here
20     }
21
22     public double solveProblem1(){
23         // add your code here
24     }
25 }

```

After you implement the first class, alter your main code to create a new object, solve the problem, and print the result. Verify that your code outputs the expected result. Take note of what inputs you used to test your code. These notes will be helpful on part 7.

For this problem, **describe what your class does** and document your work by **pasting in the code for this new class**.

Solution: This class accepts the lengths of two legs of a right triangle from user input and outputs the length of the hypotenuse using the Pythagorean theorem.

```

1  /*
2  * Title of Class: PythagoreanTheorem
3  * Author's Name: Charlotte Collard

```

```

4  * Purpose: To take two user-inputted side lengths and output the hypotenuse of
   * a right triangle.
5  *
6  * Resources:
7  *
8  */
9  import java.util.Scanner;
10
11 public class PythagoreanTheorem {
12     private double a;
13     private double b;
14
15     public PythagoreanTheorem() {
16         Scanner input = new Scanner(System.in);
17         System.out.println("Please input a your first side length: ");
18         a = input.nextDouble();
19         System.out.println("Please input a your second side length: ");
20         b = input.nextDouble();
21         input.close();
22     }
23
24     public double solvePythagoreanTheorem(){
25         double aSquare = Math.pow(a,2);
26         double bSquare = Math.pow(b,2);
27         double cSquare = Math.sqrt(aSquare + bSquare);
28         return Math.sqrt(cSquare);
29     }
30 }
31

```

7 Completing the other two classes

Now that your first problem solver is working, it is time to implement the other two. Use your first class as a template when creating the next two. Alter your main function so that the solvers will be utilized. Here is an optional way to handle the selection of problem type after storing the user's choice as the int `userInput`.

```

1  if (userInput==1) {
2      Problem1 problem = new Problem1();
3      problem.solver();
4  }
5  if (userInput==2) {
6      Problem2 problem = new Problem2();
7      problem.solver();
8  }
9  if (userInput==3) {
10     Problem3 problem = new Problem3();
11     problem.solver();
12 }

```

For this response, please provide the code for both classes.

Solution:
AreaOfCircle

```
1  /*
2  * Title of Class: AreaOfCircle
3  * Author's Name: Charlotte Collard
4  * Purpose: To take a user-inputted radius length and produce the area of a
   circle with that radius.
5  *
6  * Resources:
7  *
8  */
9  import java.util.Scanner;
10
11 public class AreaOfCircle {
12     private double r;
13
14     public AreaOfCircle() {
15         Scanner input = new Scanner(System.in);
16         System.out.println("Please input the radius length: ");
17         r = input.nextDouble();
18         input.close();
19     }
20
21     public double solveAreaOfCircle(){
22         double rSquare = Math.pow(r,2);
23         return Math.PI*rSquare;
24     }
25 }
26
```

SlopeOfLine

```
1  /*
2  * Title of Class: SlopeOfLine
3  * Author's Name: Charlotte Collard
4  * Purpose: To take two user-inputted points on the x-y plane and provide the
   slope of the line between those points.
5  *
6  * Resources:
7  *
8  */
9  import java.util.Scanner;
10
11 public class SlopeOfLine {
12     private double y1;
13     private double y2;
14     private double x1;
15     private double x2;
16
17     public SlopeOfLine() {
18         Scanner input = new Scanner(System.in);
19         System.out.println("Please input the x-coordinate of your first point:
   ");
20         x1 = input.nextDouble();
21         System.out.println("Please input the y-coordinate of your first point:
   ");
22         y1 = input.nextDouble();
23         System.out.println("Please input the x-coordinate of your second point:
   ");
24     }
25 }
26
```

```

24     x2 = input.nextDouble();
25     System.out.println("Please input the y-coordinate of your second point:
    ");
26     y2 = input.nextDouble();
27     input.close();
28 }
29
30 public double solveSlopeOfLine(){
31     double changeInY = y2 - y1;
32     double changeInX = x2 - x1;
33     return changeInY/changeInX;
34 }
35 }
36

```

8 Verifying your code

For this next part, you will develop at least three tests, where one test corresponds to each class. You may also want to add additional tests for edge cases in particular problems. To show your work, please list the test case you used and a screenshot of the console. If one of the tests fail, you do not have to edit previous sections of the writeup. Instead, correct your code and note the changes in your response to this question. Additionally, I will use your test cases and at least three more tests, when I run your code. The example below provides how to writeup your test cases, but not the screenshots.

Example:

Test 1: PASSED Inputs: 1,3,4 Output: 5

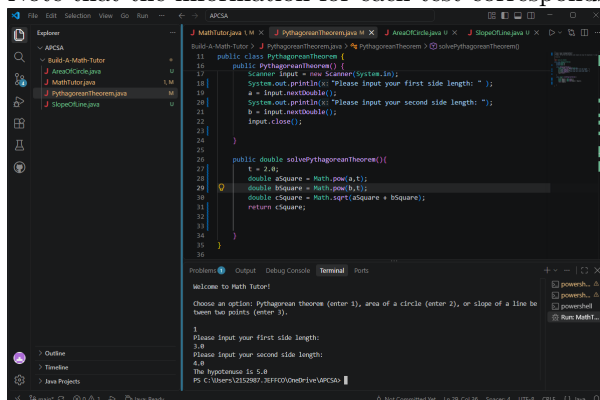
Test 2: FAILED, PASSED Originally, I had a minus sign instead of a plus sign. So, I fixed that and then it passed this test.

Inputs: 2,(3,4),(11,8) Output: 1/2

Test 3: PASSED Inputs: 3,5 Output: 78.539816

Solution:

Note that the information for each test corresponds to the screenshot directly above it.



Test 1: FAILED, PASSED

Originally, I took the square root of aSquare plus bSquare and called that cSquare, then took the square root again of cSquare. I only needed to take the square root of aSquare plus bSquare, so I

modified the code to do that instead.

Inputs: 3.0, 4.0 Output: 5.0

```

14 public class MathTutor {
15     public static void main ( String [] args ) throws IOException {
16         if ( userInput == 1 ) {
17             PythagoreanTheorem myPythagoreanTheorem = new PythagoreanTheorem();
18             double pythagoreanResult = myPythagoreanTheorem.solvePythagoreanTheorem();
19             System.out.println("The hypotenuse is " + pythagoreanResult);
20         }
21         if ( userInput == 2 ) {
22             AreaOfCircle myAreaOfCircle = new AreaOfCircle();
23             double areaOfCircleResult = myAreaOfCircle.solveAreaOfCircle();
24             System.out.println("The area is " + areaOfCircleResult);
25         }
26         if ( userInput == 3 ) {
27             SlopeOfLine mySlopeOfLine = new SlopeOfLine();
28             double slopeOfLineResult = mySlopeOfLine.solveSlopeOfLine();
29             System.out.println("The slope is " + slopeOfLineResult);
30         }
31     }
32 }

```

Terminal Output:

```

Welcome to Math Tutor!
Choose an option: Pythagorean theorem (enter 1), area of a circle (enter 2), or slope of a line (enter 3).
Please input the radius length:
3.0
The area is 28.27433882308138

```

Test 2: PASSED

Input: 3.0 Output: 28.27433882308138

```

33 Please input the x-coordinate of your first point:
1.0
Please input the y-coordinate of your first point:
2.0
Please input the x-coordinate of your second point:
3.0
Please input the y-coordinate of your second point:
4.0
The slope is 1.0

```

Test 3: PASSED

Inputs: 1.0, 2.0, 3.0, 4.0 Output: 1.0

9 Optional Challenges

Try one or more of these for additional challenge:

- Allow the user to solve multiple problems without restarting the program.
- Create five or more problem types.
- Format answers to a specific number of decimal places.
- Check for invalid inputs.
- Create helper methods to reduce repeated code.
- Display formulas before solving them.
- Generate practice problems and check user answers.

Each successful implementation should be noted here and 2 extra credit points will be rewarded per challenge.

Solution: No optional challenges were attempted.

10 Uploading Code

Make sure your code is pushed to Github. Provide a link to your code.

Solution: My code is on github at <https://github.com/ctcollard/Build-A-Math-Tutor>